MIGRATING TO A TEMPORAL SCHEMA

Paul A. Jungwirth

PG-Data 2026

5 June 2026

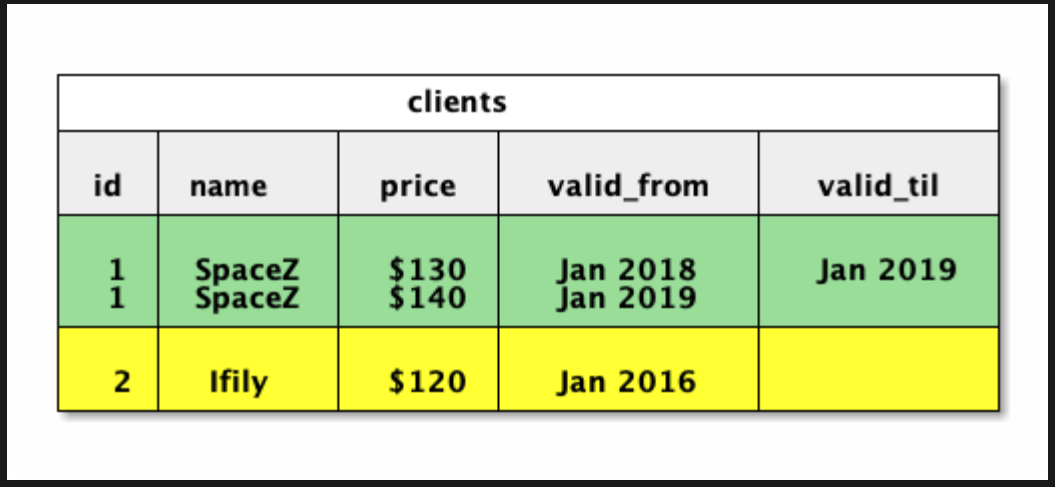
OUTLINE

- Temporal Tables
- Queries
- DML
- DDL

GOALS

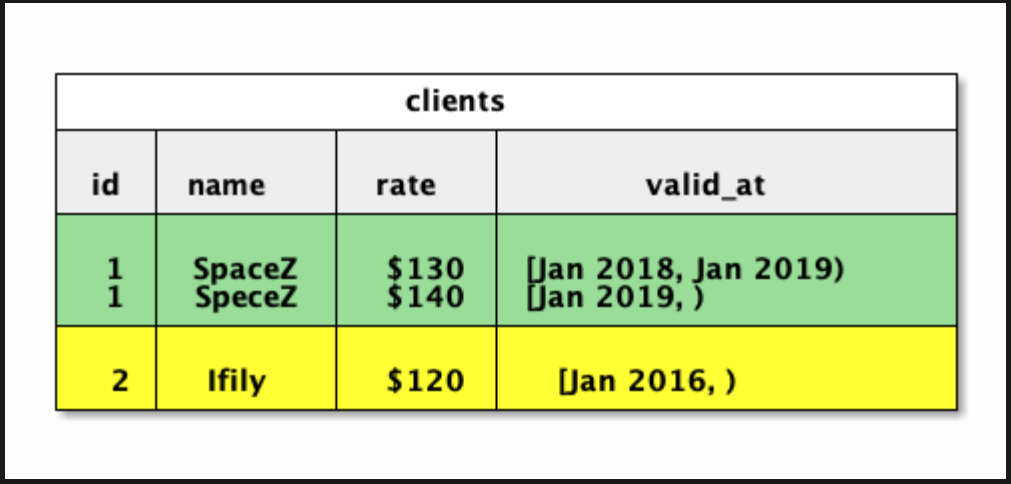
- ☒ Data Preserving
- ☒ Backwards Compatible
- ☒ Incremental Progress
- ☐ Online Migration

APPLICATION TIME



clients				
id	name	price	valid_from	valid_til
1	SpaceZ	$130	Jan 2018	Jan 2019
1	SpaceZ	$140	Jan 2019	
2	Ifily	$120	Jan 2016	

APPLICATION TIME



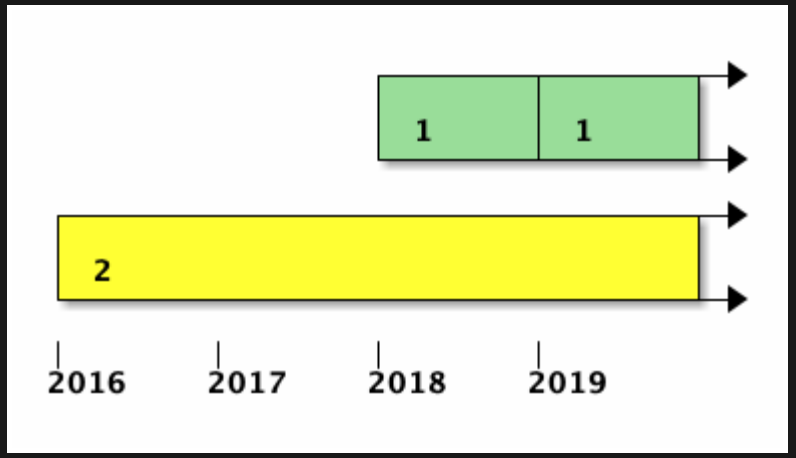
clients

id	name	rate	valid_at
1	SpaceZ	$130	[Jan 2018, Jan 2019)
1	SpeceZ	$140	[Jan 2019,)
2	Ifily	$120	[Jan 2016,)

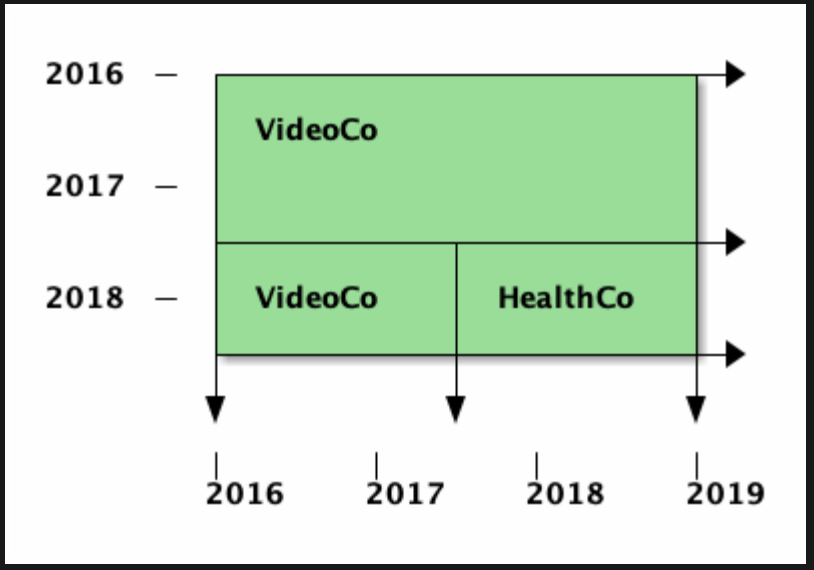
APPLICATION TIME

clients			
id	name	rate	valid_at
1	SpaceZ	$130	[Jan 2018, Jan 2019)
1	SpeceZ	$140	[Jan 2019,)
2	Ifily	$120	[Jan 2016,)
3	Nadir	$100	[Jan 2016,)
3	Nadir	$110	[Jan 2017, Jan 2018)

APPLICATION TIME



SYSTEM TIME


2016
2017
2018
VideoCo
VideoCo
HealthCo
2016
2017
2018
2019

PRIMARY KEYS

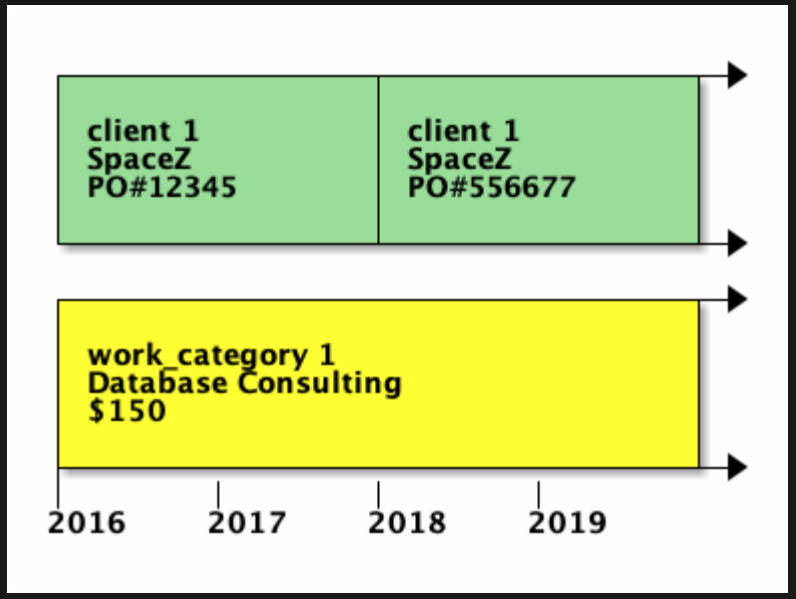
```
-- old:
CONSTRAINT legacy_pk EXCLUDE
  (id WITH =, valid_at WITH &&)

-- new:
CONSTRAINT shiny_pk PRIMARY KEY
  (id, valid_at WITHOUT OVERLAPS)
```

FOREIGN KEYS

```
CONSTRAINT fk_work_categories_on_client_id
  FOREIGN KEY (client_id, PEROID valid_at)
  REFERENCES clients (id, PERIOD valid_at)
```

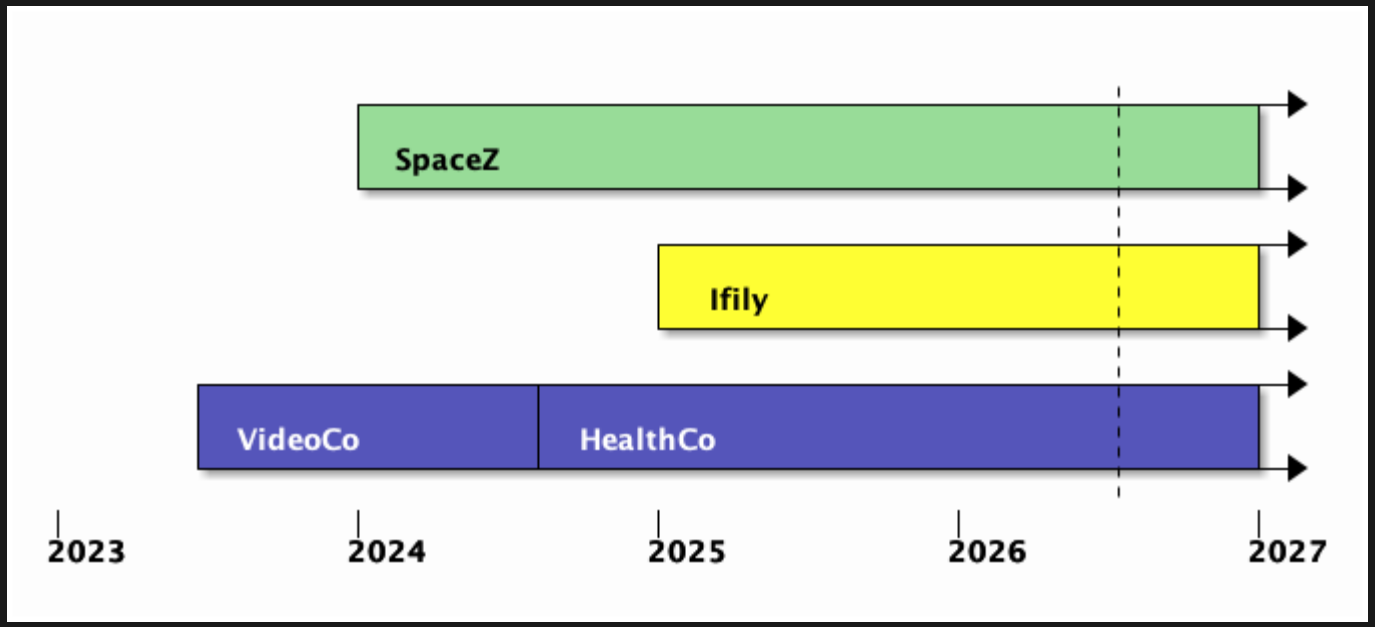
FOREIGN KEYS



FOREIGN KEYS

referencing	referenced	supported?
normal	normal	☒
temporal	temporal	☒
temporal	normal	☒
normal	temporal	☐

QUERIES



QUERIES

```
SELECT  * EXCEPT valid_at
FROM    t
WHERE   valid_at @> current_time;
```

QUERIES

$$\sigma(R \boxtimes S) = \sigma(R) \times \sigma(S)$$

We now examine whether the valid-time algebra is in some sense a consistent extension of the snapshot algebra. An algebra is said to *reduce* to the snapshot algebra if taking a snapshot of the result of applying a valid-time operator on one or two valid-time relations is identical to the result of applying the analogous snapshot operator to the snapshots (at the same times) of the valid-time relation(s). Because the temporal algebra allows tuples that contain attributes of differing timestamps, it satisfies this property only through the introduction of distinguished nulls when taking snapshots. We avoid this problem by proving a weaker property: we restrict reducibility to operations on valid-time relations that have identical timestamps for all of their attributes, termed *homogeneous relations* [Gad88a].

Theorem 4 *The valid-time operators* $\hat{\cup}$, $\hat{-}$, $\hat{\times}$, $\hat{\sigma}$, *and* $\hat{\pi}$ *reduce to their snapshot counterparts when their arguments are homogeneous.*

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

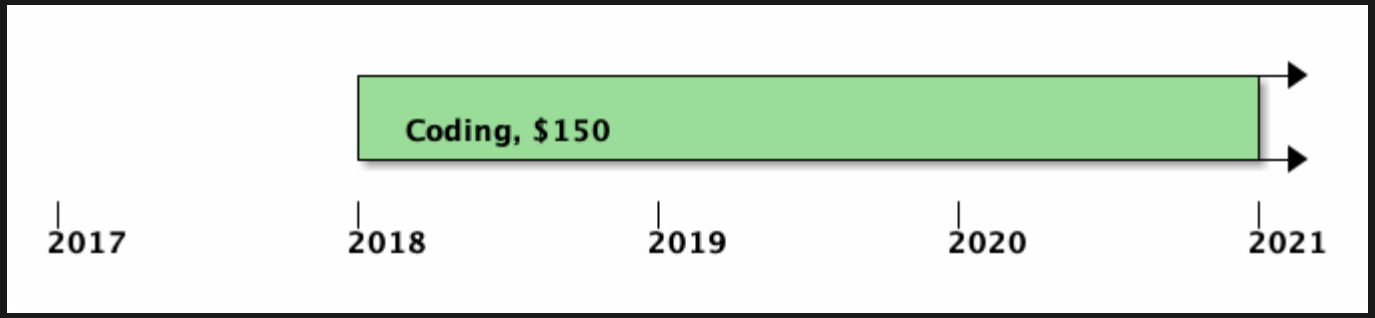
VIEWS

```sql
SELECT  id, name, slug,
        ..., position,
        -- created_at: The very first version's lower(valid_at):
        -- range_agg does the right thing for NULL endpoints, unlike MIN.
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(valid_at) AS updated_at,
        -- deleted_at: The very last version's upper(valid_at), which is usuall
        -- range_agg does the right thing for NULL endpoints, unlike MAX.
        (SELECT upper(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS deleted_at,
        archived
FROM    temporal.clients
WHERE   valid_at @> current_timestamp at time zone 'UTC'
```

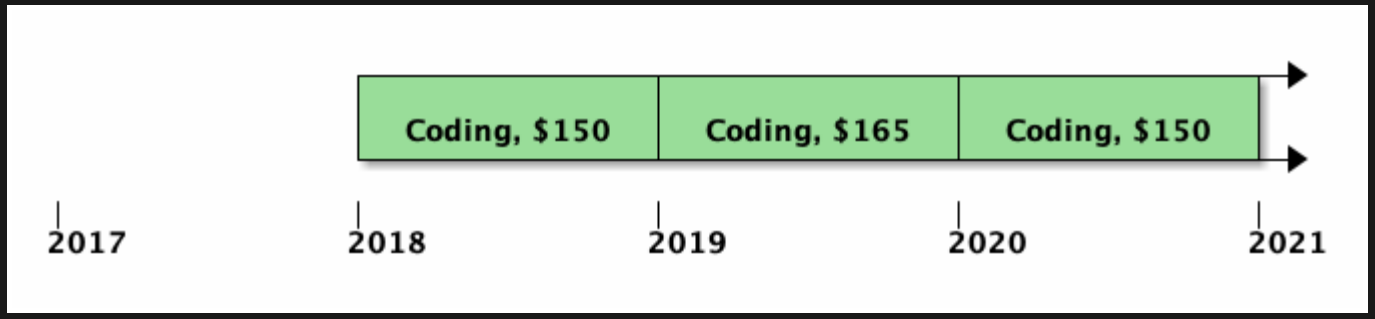
UPDATE

```
UPDATE work_categories
  FOR PORTION OF valid_at
  FROM '2019-01-01' TO '2020-01-01'
  SET rate = 1.10 * rate;
  WHERE client_id = 5;
```

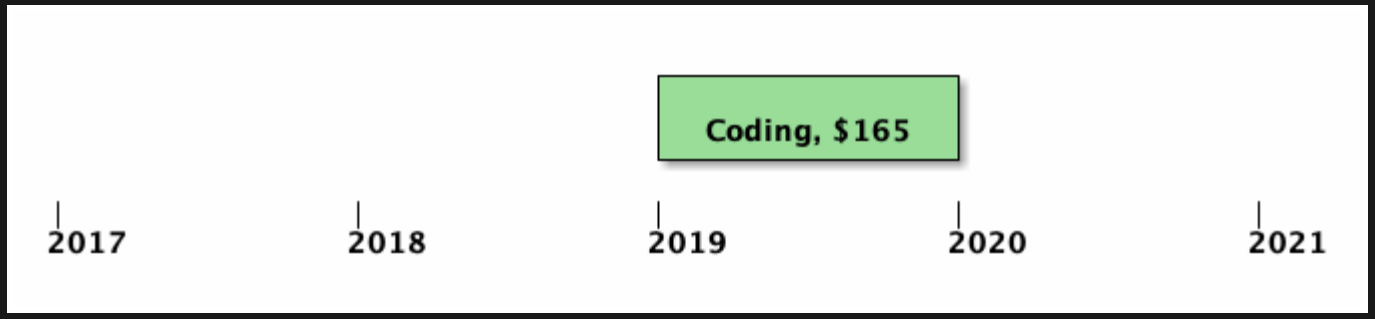
UPDATE



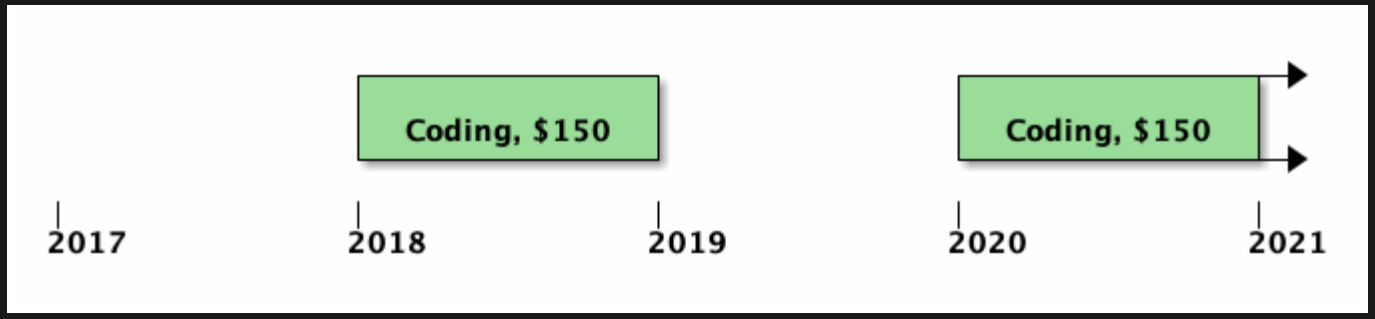
UPDATE



UPDATE



UPDATE



UPDATE

```
 1 CREATE FUNCTION update_work_categories_view() RETURNS trigg
 2 LANGUAGE plpgsql AS
 3 $$
 4 BEGIN
 5   UPDATE temporal.work_categories
 6     FOR PORTION OF valid_at
 7       FROM current_timestamp AT TIME ZONE 'UTC'
 8       TO NULL
 9     SET client_id = NEW.client_id,
10         name      = NEW.name,
11         rate      = NEW.rate
12     WHERE id = NEW.id;
13   RETURN NEW; -- Enables RETURNING
14 END;
15 $$;
```

UPDATE

```
1  CREATE FUNCTION update_work_categories_view() RETURNS trigg
2  LANGUAGE plpgsql AS
3  $$
4  BEGIN
5    UPDATE temporal.work_categories
6      FOR PORTION OF valid_at
7        FROM current_timestamp AT TIME ZONE 'UTC'
8        TO NULL
9      SET client_id = NEW.client_id,
10         name      = NEW.name,
11         rate      = NEW.rate
12     WHERE id = NEW.id;
13   RETURN NEW; -- Enables RETURNING
14 END;
15 $$;
```

UPDATE

```
1  CREATE FUNCTION update_work_categories_view() RETURNS trigg
2  LANGUAGE plpgsql AS
3  $$
4  BEGIN
5    UPDATE temporal.work_categories
6      FOR PORTION OF valid_at
7        FROM current_timestamp AT TIME ZONE 'UTC'
8        TO NULL
9      SET client_id = NEW.client_id,
10         name      = NEW.name,
11         rate      = NEW.rate
12     WHERE id = NEW.id;
13   RETURN NEW; -- Enables RETURNING
14 END;
15 $$;
```

UPDATE

```
CREATE TRIGGER trg_update_work_categories
  INSTEAD OF UPDATE ON work_categories
  FOR EACH ROW
  EXECUTE FUNCTION update_work_categories_view();
```

DELETE

```
DELETE FROM clients
  FOR PORTION OF valid_at
    FROM current_timestamp AT TIME ZONE 'UTC'
    TO NULL
  WHERE name = 'Nadir';
```

INSERT

```
 1 CREATE FUNCTION insert_work_categories_view()
 2 RETURNS trigger
 3 LANGUAGE plpgsql
 4 AS
 5 $$
 6 BEGIN
 7   INSERT INTO temporal.work_categories
 8     (client_id, name, rate, valid_at) VALUES
 9     (NEW.client_id,  NEW.name, NEW.rate,
10     tsrange(current_timestamp AT TIME ZONE 'UTC', NULL))
11     RETURNING id INTO NEW.id;
12   RETURN NEW;
13 END;
14 $$;
```

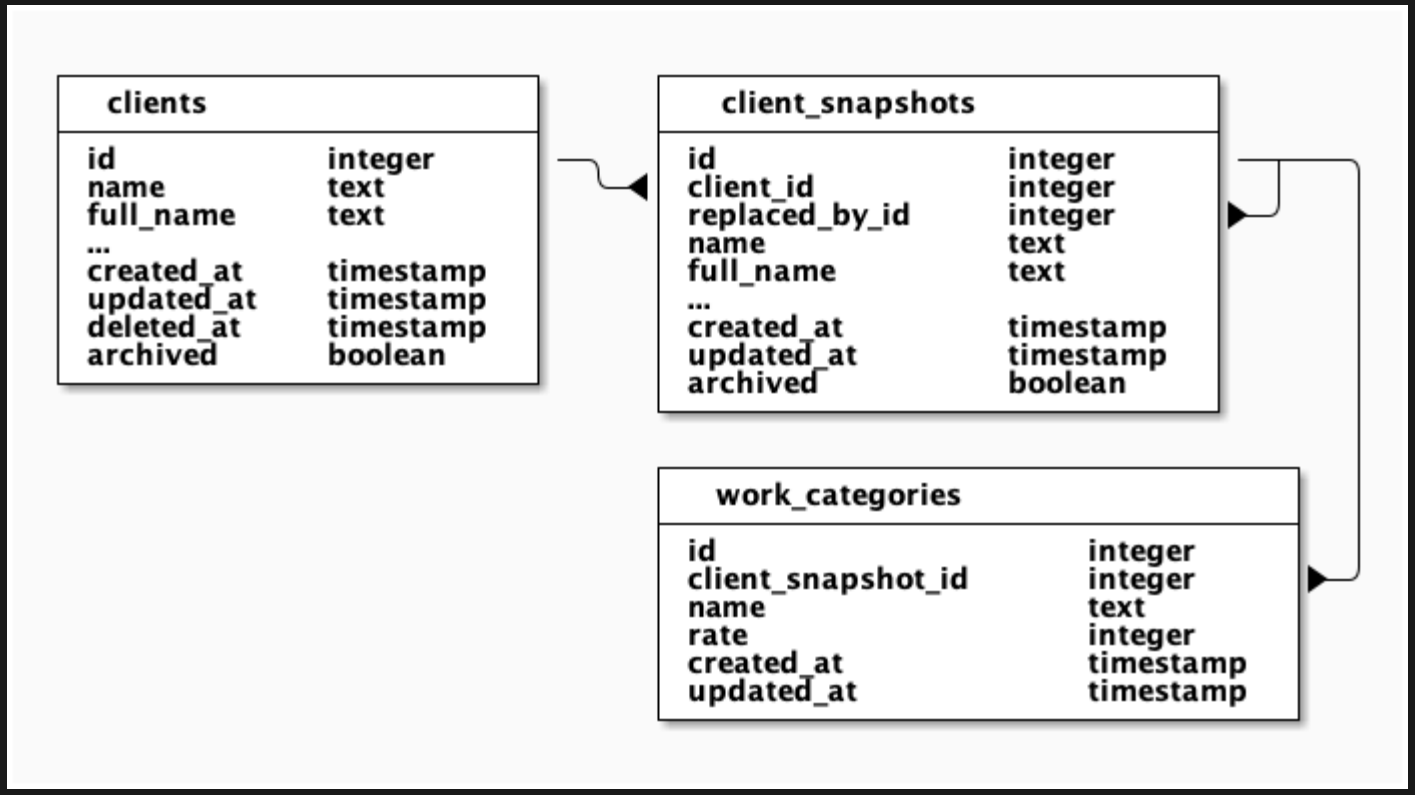
INSERT

```
 1 CREATE FUNCTION insert_work_categories_view()
 2 RETURNS trigger
 3 LANGUAGE plpgsql
 4 AS
 5 $$
 6 BEGIN
 7   INSERT INTO temporal.work_categories
 8     (client_id, name, rate, valid_at) VALUES
 9     (NEW.client_id,  NEW.name, NEW.rate,
10     tsrange(current_timestamp AT TIME ZONE 'UTC', NULL))
11     RETURNING id INTO NEW.id;
12   RETURN NEW;
13 END;
14 $$;
```

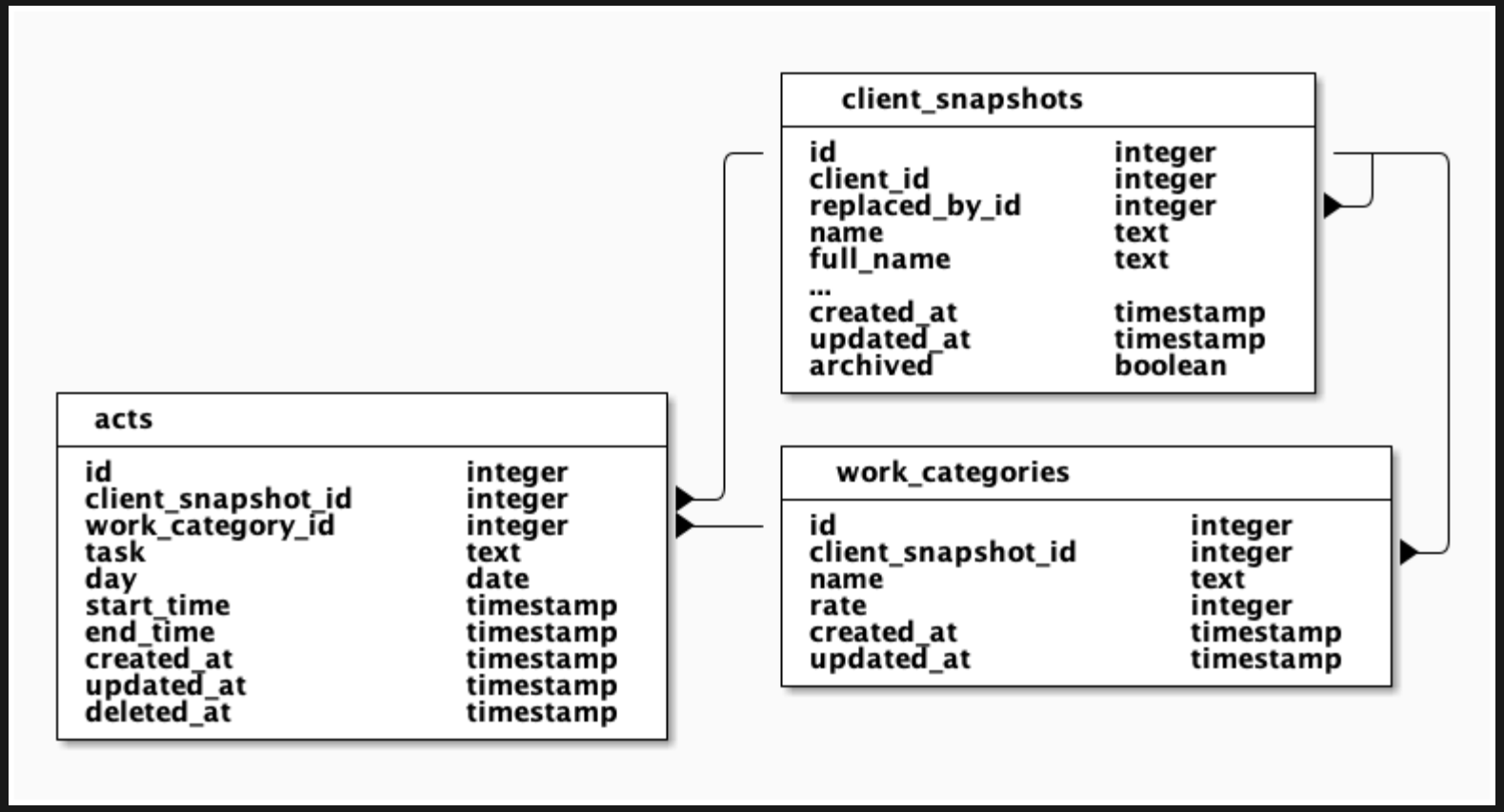
INSERT

```
 1 CREATE FUNCTION insert_work_categories_view()
 2 RETURNS trigger
 3 LANGUAGE plpgsql
 4 AS
 5 $$
 6 BEGIN
 7   INSERT INTO temporal.work_categories
 8     (client_id, name, rate, valid_at) VALUES
 9     (NEW.client_id,  NEW.name, NEW.rate,
10     tsrange(current_timestamp AT TIME ZONE 'UTC', NULL))
11     RETURNING id INTO NEW.id;
12   RETURN NEW;
13 END;
14 $$;
```

LEGACY TABLES



LEGACY TABLES



MIGRATION PLAN

- Clean up bad data.
  - Add more constraints.
- Create `temporal` schema with tables.
- Move data over from the old tables.
- Staging:
  - `temporal.clients` with `client_snapshots` view
  - remove `client_snapshots`
  - `temporal.work_categories` with view
  - remove `work_categories` view

INCREMENTAL PROGRESS

1. Add a dummy `valid_at` to the non-temporal table.
2. Put a traditionally-unique column on the temporal table.
3. Define a "loose" foreign key function.
4. Create separate id tables.

LOOSE FK

```
 1 CREATE OR REPLACE FUNCTION add_loose_foreign_key(
 2   from_table regclass,
 3   fk_column  text,
 4   to_table   regclass,
 5   pk_column  text DEFAULT 'id',
 6   name       text DEFAULT NULL
 7 ) RETURNS void LANGUAGE plpgsql AS $fn$
 8 DECLARE
 9   v_suffix   text := loose_fk_trigger_suffix(from_table, fk
10   v_ref_name text := 'loose_fk_ref_' || v_suffix;
11   v_inv_name text := 'loose_fk_inv_' || v_suffix;
12 BEGIN
13   EXECUTE format(
14     'CREATE CONSTRAINT TRIGGER %1$I
15        AFTER INSERT OR UPDATE OF %2$I ON %3$s
```

LOOSE FK

```
11    v_inv_name text := 'loose_fk_inv_' || v_suffix;
12  BEGIN
13    EXECUTE format(
14      'CREATE CONSTRAINT TRIGGER %1$I
15         AFTER INSERT OR UPDATE OF %2$I ON %3$s
16         FROM %4$s
17         NOT DEFERRABLE INITIALLY IMMEDIATE
18         FOR EACH ROW
19         EXECUTE PROCEDURE loose_fk_check_ref(%5$L, %6$L, %7$
20      v_ref_name, fk_column, from_table, to_table,
21      fk_column, to_table::text, pk_column
22    );
23
24    EXECUTE format(
```

LOOSE FK

```
19          EXECUTE PROCEDURE loose_fk_check_ref(%5$L, %6$L, %7$
20        v_ref_name, fk_column, from_table, to_table,
21        fk_column, to_table::text, pk_column
22      );
23
24      EXECUTE format(
25        'CREATE CONSTRAINT TRIGGER %1$I
26           AFTER UPDATE OF %2$I OR DELETE ON %3$s
27           FROM %4$s
28           NOT DEFERRABLE INITIALLY IMMEDIATE
29           FOR EACH ROW
30           EXECUTE PROCEDURE loose_fk_check_inv(%5$L, %6$L, %7$
31        v_inv_name, pk_column, to_table, from_table,
32        from_table::text, fk_column, pk_column
33      );
```

LOOSE FK

```
 1 CREATE OR REPLACE FUNCTION loose_fk_check_ref()
 2 RETURNS trigger LANGUAGE plpgsql AS $fn$
 3 DECLARE
 4   v_fk_column text := TG_ARGV[0];
 5   v_to_table  text := TG_ARGV[1];
 6   v_pk_column text := TG_ARGV[2];
 7   v_is_null   boolean;
 8   v_found     integer;
 9   v_fk_value  text;
10 BEGIN
11   -- MATCH SIMPLE: a NULL fk passes the constraint.
12   EXECUTE format('SELECT ($1).%I IS NULL', v_fk_column)
13     USING NEW INTO v_is_null;
14   IF v_is_null THEN RETURN NULL; END IF;
15
```

LOOSE FK

```
 6   v_pk_column text := TG_ARGV[2];
 7   v_is_null   boolean;
 8   v_found     integer;
 9   v_fk_value  text;
10 BEGIN
11   -- MATCH SIMPLE: a NULL fk passes the constraint.
12   EXECUTE format('SELECT ($1).%I IS NULL', v_fk_column)
13     USING NEW INTO v_is_null;
14   IF v_is_null THEN RETURN NULL; END IF;
15
16   -- FOR KEY SHARE matches built-in RI: lock the referenced
17   -- concurrent UPDATE-of-key or DELETE blocks until we com
18   EXECUTE format(
19     'SELECT 1 FROM %1$s t
```

LOOSE FK

```
12    EXECUTE format('SELECT ($1).%I IS NULL', v_fk_column)
13      USING NEW INTO v_is_null;
14    IF v_is_null THEN RETURN NULL; END IF;
15
16    -- FOR KEY SHARE matches built-in RI: lock the referenced
17    -- concurrent UPDATE-of-key or DELETE blocks until we com
18    EXECUTE format(
19      'SELECT 1 FROM %1$s t
20       WHERE t.%2$I = ($1).%3$I FOR KEY SHARE OF t',
21      v_to_table, v_pk_column, v_fk_column
22    ) USING NEW INTO v_found;
23
24    IF v_found IS NULL THEN
25      v_fk_value := to_jsonb(NEW) ->> v_fk_column;
26      RAISE foreign_key_violation USING
```

ID TABLES


```sql
CREATE TABLE temporal.single_clients (id integer PRIMARY KEY);

CREATE FUNCTION trg_generate_temporal_client_id()
RETURNS trigger LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.id IS NULL THEN
    NEW.id := nextval(
      pg_get_serial_sequence('temporal.clients', 'id'));
  END IF;
  INSERT INTO temporal.single_clients (id) VALUES (NEW.id)
    ON CONFLICT DO NOTHING;
  RETURN NEW;
```

```
END;
$$;
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE TABLE temporal.clients (
  id integer NOT NULL GENERATED BY DEFAULT AS IDENTITY
    /* REFERENCES temporal.single_clients (id) */,
  valid_at tsrange NOT NULL,
  ...,
  position integer NOT NULL DEFAULT 0,
  archived boolean NOT NULL DEFAULT false,
  client_snapshot_id SERIAL NOT NULL,
  CONSTRAINT clients_pkey PRIMARY KEY (id, valid_at WITHOUT OVERLAPS),
  CONSTRAINT clients_slug_key UNIQUE (slug, valid_at WITHOUT OVERLAPS)
  -- acts_as_list will trip this if we check
  -- in the middle of a transaction (same as for non-temporal):
  CONSTRAINT clients_position_key UNIQUE (position, valid_at WITHOUT O
    DEFERRABLE INITIALLY DEFERRED,
  CONSTRAINT clients_client_snapshot_id_key UNIQUE (client_snapshot_id
);
EOQ
```

CLIENTS

```
CREATE OR REPLACE FUNCTION trg_generate_temporal_client_snapsh
RETURNS trigger
LANGUAGE plpgsql
AS
$$
BEGIN
  SELECT nextval(
           pg_get_serial_sequence('temporal.clients',
                                  'client_snapshot_id'))
    INTO NEW.client_snapshot_id;
  RETURN NEW;
END;
$$;
```

CLIENTS

```
 1 INSERT INTO temporal.clients
 2 (id, ..., valid_at, name, position, client_snapshot_id)
 3 SELECT  cs.client_id, cs.name, ...,
 4         tsrange(
 5           cs.created_at,
 6           lead(cs.created_at)
 7             OVER (PARTITION BY cs.client_id ORDER BY cs.created_at)),
 8         dense_rank()
 9           OVER (ORDER BY c.position
10                 ROWS UNBOUNDED PRECEDING EXCLUDE TIES)
11           AS position,
12         cs.id
13 FROM    client_snapshots cs
14 JOIN    clients c ON c.id = cs.client_id;
```

CLIENTS

```
 1 INSERT INTO temporal.clients
 2 (id, ..., valid_at, name, position, client_snapshot_id)
 3 SELECT  cs.client_id, cs.name, ...,
 4         tsrange(
 5           cs.created_at,
 6           lead(cs.created_at)
 7             OVER (PARTITION BY cs.client_id ORDER BY cs.created_at)),
 8         dense_rank()
 9           OVER (ORDER BY c.position
10                 ROWS UNBOUNDED PRECEDING EXCLUDE TIES)
11           AS position,
12         cs.id
13 FROM    client_snapshots cs
14 JOIN    clients c ON c.id = cs.client_id;
```

CLIENTS

```
 1 INSERT INTO temporal.clients
 2 (id, ..., valid_at, name, position, client_snapshot_id)
 3 SELECT  cs.client_id, cs.name, ...,
 4         tsrange(
 5           cs.created_at,
 6           lead(cs.created_at)
 7             OVER (PARTITION BY cs.client_id ORDER BY cs.created_at)),
 8         dense_rank()
 9           OVER (ORDER BY c.position
10                 ROWS UNBOUNDED PRECEDING EXCLUDE TIES)
11           AS position,
12         cs.id
13 FROM    client_snapshots cs
14 JOIN    clients c ON c.id = cs.client_id;
```

CLIENTS

```
 1 INSERT INTO temporal.clients
 2 (id, ..., valid_at, name, position, client_snapshot_id)
 3 SELECT  cs.client_id, cs.name, ...,
 4         tsrange(
 5           cs.created_at,
 6           lead(cs.created_at)
 7             OVER (PARTITION BY cs.client_id ORDER BY cs.created_at)),
 8         dense_rank()
 9           OVER (ORDER BY c.position
10                 ROWS UNBOUNDED PRECEDING EXCLUDE TIES)
11           AS position,
12         cs.id
13 FROM    client_snapshots cs
14 JOIN    clients c ON c.id = cs.client_id;
```

CLIENTS

```
 1 INSERT INTO temporal.clients
 2 (id, ..., valid_at, name, position, client_snapshot_id)
 3 SELECT  cs.client_id, cs.name, ...,
 4         tsrange(
 5           cs.created_at,
 6           lead(cs.created_at)
 7             OVER (PARTITION BY cs.client_id ORDER BY cs.created_at)),
 8         dense_rank()
 9           OVER (ORDER BY c.position
10                 ROWS UNBOUNDED PRECEDING EXCLUDE TIES)
11           AS position,
12         cs.id
13 FROM    client_snapshots cs
14 JOIN    clients c ON c.id = cs.client_id;
```

client_snapshots

```sql
SELECT  client_snapshot_id AS id,
        id AS client_id,
        lead(client_snapshot_id)
          OVER (PARTITION BY id ORDER BY id)
          AS replaced_by_id,
        name,
        ...,
        lower(valid_at) AS created_at,
        -- In practice the old data often has created_at <> up
        -- because we set replaced_by_id,
        -- but the app doesn't care if we lie about it:
        lower(valid_at) AS updated_at,
FROM    temporal.clients
```

work_categories

```
SELECT  original_id AS id,
        c.client_snapshot_id,
        wc.name, wc.rate, wc.position,
        -- created_at: The very first version's lower(valid_at
        -- range_agg does the right thing for NULL endpoints,
        (SELECT lower(range_agg(valid_at))
          FROM  temporal.clients c2
          WHERE c2.id = clients.id) AS created_at,
        lower(wc.valid_at) AS updated_at,
        wc.retainer_hours
FROM    temporal.work_categories wc
JOIN    temporal.clients c
ON      c.id = wc.client_id AND c.valid_at && wc.valid_at
WHERE   wc.valid_at @> current_timestamp at time zone 'UTC'
```

THANKS!

https://github.com/pjungwir/migrating-to-a-temporal-schema

REFERENCES

- Richard Snodgrass. "An Overview of TQuel." *Temporal Databases: Theory, Design, and Implementation*, 1993.
- Anton Dignös, Michael H. Böhlen, Johann Gamper. "Temporal Alignment." SIGMOD, 2012. https://files.ifi.uzh.ch/boehlen/Papers/modf174-dignoes.pdf
- Paul Jungwirth. Temporal Ops Postgres Extension. https://github.com/pjungwir/temporal_ops
- Boris Novikov. "PostgreSQL Temporal Aggregates: SUM, AVG & COUNT Across Time." Red Gate, 2024. https://www.red-gate.com/simple-talk/databases/postgresql/making-temporal-databases-work-part-2-computing-aggregates-across-temporal-versions/
- Paul Jungwirth. These slides. https://github.com/pjungwir/migrating-to-a-temporal-schema